Included Lives — Plataforma de Oportunidades PCD

1. Nome do Projeto: Included Lives
Curso: Desenvolvimento de Software Multiplataforma
2. Integrantes do Grupo: Adriana Carla Rodrigues; Gabriel Sebastião; Pâmela Roberta; Vinícius Sodré; Wallace Petterson;
Data de Entrega: 06/Novembro/2026
Linguagem Obrigatória: Java SE (versão 17+)

3. Problema Identificado

Profissionais com deficiência (PCD) deparam-se com persistentes assimetrias de informação e barreiras no acesso a vagas formais no mercado corporativo. Embora a legislação vigente (Lei nº 8.213/1991 — Lei de Cotas) estabeleça a obrigatoriedade de percentuais inclusivos em empresas, as plataformas convencionais de recrutamento limitam-se a assinalar que a oportunidade aceita pessoas com deficiência, sem qualquer especificação técnica a respeito dos recursos físicos, ergonômicos e tecnológicos disponíveis no ambiente laboral.
Essa carência gera incompatibilidade crônica durante a triagem inicial: candidatos com baixa visão inscrevem-se em cargos que não disponibilizam tecnologias assistivas (leitores de tela), pessoas com deficiência física descobrem barreiras arquitetônicas intransponíveis apenas na etapa presencial, e profissionais neurodivergentes não encontram clareza quanto a flexibilidades comunicacionais. Como consequência, observa-se alta taxa de desistência nas etapas seletivas, processos admissionais ineficientes e cumprimento puramente burocrático das políticas de diversidade.

4. Público-Alvo

- **Candidatos PCD:** Profissionais em busca de colocações compatíveis com sua formação e experiência, que necessitam de transparência quanto ao código CID atendido, adaptações ergonômicas e tecnologias assistivas disponíveis no posto de trabalho.
- **Empresas Contratantes:** Organizações que visam cumprir as metas legais de inclusão com eficiência, necessitando divulgar vagas qualificadas com especificação rigorosa de recursos de acessibilidade e realizar a gestão progressiva de seus processos seletivos.

5. Objetivo da Aplicação

Desenvolver uma solução computacional robusta baseada no paradigma de Programação Orientada a Objetos (POO), construída em linguagem Java, que formalize a mediação ética e técnica entre profissionais com deficiência e organizações inclusivas. A aplicação centraliza perfis curriculares especializados, catálogo de vagas estruturadas por suporte de acessibilidade, e mecanismos de validação transacional para candidaturas e estágios de recrutamento, operando a partir de uma interface textual em console com arquitetura em camadas desacopladas.

6. Principais Funcionalidades

- **Cadastro Corporativo e Especializado:** Registro de empresas com dados fiscais e operacionais, e de candidatos PCD com laudo, código CID, necessidades de suporte, formações acadêmicas e competências técnicas.
- **Gestão do Ciclo de Vida de Vagas:** Abertura, ativação, pausa e encerramento de anúncios de emprego com indicação mandatória de localidade, faixa salarial e infraestrutura assistiva oferecida.
- **Inscrição Controlada em Processos Seletivos:** Vinculação entre candidato e vaga com validação rigorosa de elegibilidade, impedindo candidaturas em postos inativos e bloqueando inscrições em duplicidade.
- **Progressão Linear de Etapas de Seleção:** Rastreamento e transição controlada das candidaturas pelas fases sequenciais: `INSCRITO` → `EM_ANALISE` → `ENTREVISTA` → `FINALIZADO`.
- **Consulta Filtrada no Console:** Listagem dinâmica de oportunidades ativas e busca de histórico de candidaturas por identificador.
- **Exibição Polimórfica de Perfis:** Apresentação formatada e individualizada de contas de usuário (candidato ou corporativo) por despacho dinâmico de métodos.

7. Tecnologias Utilizadas

Tecnologia / Recurso	Versão / Especificação	Finalidade no Projeto
Linguagem Java	Java SE 17+ (LTS)	Implementação integral da lógica orientada a objetos, tipos genéricos e tipagem estática.
Coleções (JCF)	`List`, `Set`, `Map`	Organização de histórico ordenado, garantia de unicidade de competências e buscas O(1).
Generics	Estruturas Parametrizadas	Construção de repositórios desacoplados e segurança de tipos em tempo de compilação.
Interface Console	`java.util.Scanner`	Interface interativa textual via terminal em conformidade com o Requisito 13 do Edital.
Versionamento	Git / GitHub	Rastreamento de commits semânticos e comprovação do desenvolvimento distribuído.

8. Descrição da Estrutura do Projeto

O código-fonte adota uma arquitetura em camadas (`Layered Architecture`), distribuindo responsabilidades funcionais em pacotes distintos sob a raiz `br.com.includedlives`, garantindo alta coesão e baixo acoplamento:

Pacote	Classes Integrantes	Responsabilidade Técnica
`app`	`Main.java`	Ponto de entrada (`main`); inicializa o menu e dispara a execução.
`interfaces`	`Autenticavel.java`, `Identificavel.java`	Contratos abstratos de autenticação e identificação única para entidades.
`model`	`Usuario`, `Candidato`, `Empresa`, `Vaga`, `Candidatura`, `Formacao`, `Habilidade`	Entidades de domínio, herança, encapsulamento e regras de composição.
`model.enums`	`StatusVaga`, `StatusCandidatura`, `TipoHabilidade`	Tipos enumerados para estados finitos e integridade de domínio.
`repository`	`IRepositorio.java`, `RepositorioMemoria.java`	Persistência genérica em memória isolando o modelo da infraestrutura.
`service`	`VagaService.java`, `CandidaturaService.java`	Camada transacional que executa e valida as Regras de Negócio (RN01 a RN05).
`exception`	`RegraNegocioException`, `VagaInativaException`, `CandidaturaDuplicadaException`	Exceções semânticas de negócio derivadas de `RuntimeException`.
`view`	`MenuConsole.java`	Interface gráfica de linha de comando com menus, leituras e captura de erros.

9. Identificação dos Conceitos de POO Utilizados

O sistema foi concebido para atender integralmente aos requisitos técnicos obrigatórios do edital, assegurando legitimidade conceitual em cada decisão de projeto:

9.1. Classes de Domínio (Requisito 5.1)

Foram implementadas **7 classes de domínio representativas**: `Usuario`, `Candidato`, `Empresa`, `Vaga`, `Candidatura`, `Formacao` e `Habilidade`. Classes técnicas como `Main`, `MenuConsole` ou repositórios foram explicitamente desconsideradas dessa contagem.

9.2. Encapsulamento (Requisito 5.2)

Todos os atributos das classes de domínio são estritamente **privados (`private`)**. Não há concessão de métodos setters arbitrários; alterações de estado realizam-se por métodos de negócio (`pausar()`, `encerrar()`, `avancarEtapa()`). Além disso, coleções internas retornam visualizações imutáveis (`Collections.unmodifiableList` e `Collections.unmodifiableSet`), impedindo adulterações externas.

9.3. Composição e Agregação (Requisito 5.3)

- **Composição Forte:** A classe `Candidato` compõe objetos `Formacao` e `Habilidade`. Estes elementos dependem da existência do candidato e são gerenciados estritamente através dele.
- **Agregação:** A classe `Empresa` agrega instâncias de `Vaga` em uma coleção. As vagas são publicadas pela organização, mas mantêm independência estrutural nas operações de candidatura.

- **Associação com Atributos:** A classe `Candidatura` associa instâncias ativas de `Candidato` e `Vaga`, agregando dados próprios de controle (data/hora e status da fase seletiva).

9.4. Herança (Requisito 6)

A hierarquia é encabeçada pela classe abstrata `Usuario`, que centraliza atributos essenciais de autenticação, contato e localização física (cidade e UF). Dela derivam as subclasses especializadas `Candidato` e `Empresa`. A herança é legítima e justificada pelo compartilhamento de contratos de login e presença territorial no ecossistema da plataforma.

9.5. Polimorfismo Dinâmico (Requisito 7)

A superclasse abstrata `Usuario` declara o método abstrato `public abstract String exibirPerfil();`. As subclasses `Candidato` e `Empresa` sobrescrevem este método com lógicas totalmente distintas de exibição. No menu interativo (Opção 4), o sistema itera sobre uma lista heterogênea `List<Usuario>` invocando `u.exibirPerfil()`: a JVM executa a ligação tardia (*dynamic dispatch*) invocando a versão adequada em tempo de execução sem condicionais manuais ou checagens de tipo (`instanceof`).

9.6. Interfaces Java (Requisito 8)

- `Autenticavel`: Define o contrato `boolean autenticar(String email, String senha);`, implementado diretamente na classe base `Usuario` e herdado por seus subtipos.
- `Identificavel<ID>`: Contrato genérico que padroniza o acesso ao identificador primário de qualquer entidade persistível, viabilizando o desacoplamento do repositório.

9.7. Coleções / Java Collections Framework (Requisito 9)

- `List<E>`: Utilizado para preservar a ordem sequencial de inserção nas formações de um candidato, nas vagas publicadas por uma empresa e no histórico das candidaturas.
- `Set<E>`: Utilizado na coleção `Set<Habilidade>` do candidato. Combinado à sobrescrita de `equals()` e `hashCode()` na classe `Habilidade`, o sistema impede matematicamente que a mesma competência seja cadastrada duas vezes para o mesmo profissional.
- `Map<K, V>`: Empregado no repositório de memória através de `LinkedHashMap<ID, T>`, assegurando recuperação e consultas diretas em tempo constante *O(1)* a partir da chave identificadora.

9.8. Generics (Requisito 10)

A camada de persistência foi modelada através da interface parametrizada `IRepositorio<T, ID>` e da classe `RepositorioMemoria<T extends Identificavel<ID>, ID>`. O uso de tipos genéricos proporciona reuso estrito de código e segurança de tipos em tempo de compilação (*type safety*), eliminando a necessidade de conversões forçadas de tipo (*casts*) e centralizando rotinas de busca e remoção.

9.9. Tratamento de Exceções e Regras de Negócio (Requisitos 11 e 16)

O projeto implementa uma hierarquia semântica de exceções de domínio derivadas de `RuntimeException`: a superclasse `RegraNegocioException` e suas especializações `VagaInativaException` e `CandidaturaDuplicadaException`. As 5 regras de negócio requeridas foram rigorosamente implementadas:

Regra	Descrição da Regra de Negócio	Classe e Mecanismo de Disparo
RN01	**Bloqueio em Vaga Inativa:** Inscrições só são aceitas se a vaga estiver com status `ATIVA`. Vagas pausadas ou encerradas são recusadas imediatamente.	`CandidaturaService.java` dispara `VagaInativaException`.
RN02	**Unicidade de Inscrição:** Um candidato PCD não pode se inscrever mais de uma vez na mesma oportunidade de trabalho.	`CandidaturaService.java` dispara `CandidaturaDuplicadaException`.
RN03	**Capacidade Mínima de Contratação:** Uma vaga não pode ser registrada com quantidade de postos inferior ou igual a zero (`quantVaga ≤ 0`).	`VagaService.java` dispara `RegraNegocioException`.
RN04	**Transição Progressiva de Etapas:** O avanço da seleção ocorre em sequência estrita (`INSCRITO` → `EM_ANALISE` → `ENTREVISTA` → `FINALIZADO`).	Método `avancarEtapa()` na entidade `Candidatura.java`.
RN05	**Privacidade no Banco de Talentos:** Candidatos com o campo `disponivelBanco = false` são suprimidos da listagem de busca ativa visível para recrutadores.	Filtro de visibilidade implementado em métodos de consulta no `Candidato.java`.